

OptiComp2

Operator manual

Control software for the angle-resolved (VAR) spectrophotometer

Haoxiang Zhang

August 2026 · instrument PC: C:\OptiComp2 · Python 3.9

This manual covers day-to-day operation of OptiComp2: safety, start-up, the graphical interface, running a measurement, analysis, the command-line tools, and recovery. Read **Section 2 (Safety)** before touching the instrument. The original OptiComp program must not be running at the same time — the serial bus and the spectrometer are exclusive resources.

Contents

1	Introduction	3
2	Safety and hardware cautions	3
3	Installation and layout	4
4	Start-of-day and post-incident procedure	4
4.1	A. Fibre and detector arm (lamp off)	4
4.2	B. USB soft-restart check (once, with someone present)	5
4.3	C. Lamp and motion cycle test	5
4.4	D. Shutdown	5
5	The graphical interface	5
5.1	Pages	5
5.2	Status bar and the emergency shutter	5
5.3	Keyboard shortcuts	5
6	Running a measurement	6
6.1	Connect	6
6.2	Reference calibration (with auto integration time)	6
6.3	Dark, single angle, scan and double beam	6
7	Analysis	6
8	Command-line tools	7
9	Recovery and troubleshooting	8
10	Data and file conventions	8
A	Geometry and configuration reference	9

1 Introduction

OptiComp2 drives an angle-resolved spectrophotometer that measures the variable-angle reflectance (VAR) of a sample as a function of wavelength and incidence angle, in two linear polarisations (S and P). The instrument comprises:

- A **Thorlabs Elliptec** rotation-stage assembly on one serial bus (COM4), with four addressed modules: a shutter (address 0), a polariser (address 1), the **detector arm** (address 2, which carries the collection fibre), and the **sample stage** (address 3).
- A **B&W Tek** fibre spectrometer accessed through `BWTEKUSB.dll`.
- A broadband lamp and an integrating sphere / detector optic fed by the collection fibre.

A measurement combines a *sample* session and a *reference* session (a diffuse white board, or silicon as a Fresnel standard). Reflectance is computed by double-beam substitution so that lamp and system throughput cancel. OptiComp2 reproduces the original instrument geometry and measurement model and adds the verification, calibration and recovery described in the change list.

2 Safety and hardware cautions

These are the rules that protect the instrument and the data. They are enforced in software where possible, but the operator is the last line of defence.

Caution

Never home the detector arm (module 2) casually. It carries the collection fibre; homing it can wind and snap the fibre. `home("2")` is refused unless `force=True`, and the raw command channel refuses to send `ho` to a protected address. The arm is moved only by absolute positioning to its 44° zero. If it must be re-homed, do it only with the fibre slack and someone watching (Section 4).

Caution

Do not replug USB on the instrument hub while the arm is away from zero. The Elliptec bus board (ELLB) is USB-powered and shares a hub with the spectrometer. *Any* power cycle on that hub makes all rotation modules auto-home on power-up: the polariser and sample stage snap to 0° and the detector arm attempts a home it may stall on (GS02), losing its zero. Use `tools\usb.reset.py` (a PnP restart of the spectrometer only) instead of a physical replug.

Caution

Never hard-kill a running script. Closing the window or using `taskkill` skips the cleanup that closes the shutter. To stop an unattended script use the `STOP` file (Section 8). If a script was already hard-killed, immediately run `py tools\shutter.close.py` to close and verify the shutter.

Software safeguards that back these rules:

- The stage-state record (`data\stage_state.json`) is compared on every connect and script start; a position jump $> 0.5^\circ$, a non-zero status code or a changed speed is flagged, and unattended scripts abort on an anomaly.
- A blocked home (GS02) is never retried automatically; a stalled relative move is retried as an absolute move to the original target; a reply line cut by the read timeout is discarded, never decoded.

- The Runner closes the shutter whenever a sequence fails or is aborted, and the GUI closes it on exit; auto integration time *aborts* rather than using a non-converged value.

3 Installation and layout

OptiComp2 runs on the instrument PC under Python 3.9; it needs `numpy`, `pyserial`, `matplotlib` and `Tk` (bundled with Python). The spectrometer USB recovery uses `pnputil`, which requires an elevated process — the launcher self-elevates.

Location	Contents
<code>hw/</code>	Hardware layers: <code>elliptec.py</code> (bus protocol), <code>bwtek.py</code> (spectrometer), <code>stagestate.py</code> (motor-state record), <code>config.py</code> (geometry, limits, speeds).
<code>tools/</code>	<code>manual_gui.py</code> (the GUI), <code>sequence.py</code> (step builders + Runner), and the command-line tools (Section 8).
<code>analysis/</code>	<code>var.py</code> (reflectance) and <code>standards.py</code> (silicon, BK7, constant).
<code>tests/</code>	Unit tests: <code>py -m unittest discover -s tests</code> .
<code>data/</code> , <code>standards/</code>	Measurement sessions and reference tables (never overwritten by a code deployment).

Launch the GUI by double-clicking `run_manual_gui.bat` (it requests administrator rights — click **Yes** on the UAC dialog) or from a shell:

```
py tools\manual_gui.py
```

To try the interface without any hardware, use the offline demo: `py tools\manual_gui.py --demo`. It never opens the serial port or the DLL.

4 Start-of-day and post-incident procedure

Follow this after any bus power event (a USB replug, a power blip) or at the start of a measurement day. It restores the stage reference safely.

4.1 A. Fibre and detector arm (lamp off)

1. Look at the fibre: the run is slack and not wound around the detector arm.
2. Read-only report, moves no motor: `py tools\restore_stages.py`
Expect: module 2 reports GS02 if its last auto-home failed (zero lost), the polariser near 0°, the sample stage near 103°, the shutter closed.
3. With the fibre slack and someone watching the arm, restore the reference: `py tools\restore_stages.py --safe --arm`
Type **YES** to confirm. The script parks the polariser/sample stage, sets the arm speed to 50 %, homes the arm and parks it at 44°, and writes the baseline `stage_state.json`. If the arm stalls again while homing (GS02), cut ELLB power immediately and check the fibre — do *not* re-home.

4.2 B. USB soft-restart check (once, with someone present)

In an administrator shell: `py tools\usb_reset.py`

It does a PnP restart of the spectrometer (equivalent to a replug, but only the spectrometer device) and reads a probe spectrum. Watch that the arm does not move and the state shows no anomaly. Later, when the DLL hangs (read -99 , device count 0), the GUI's **Recover** button and the monitor and cycle tools call the same logic automatically.

4.3 C. Lamp and motion cycle test

Turn on the lamp and warm up for ≥ 30 min with the shutter closed. Mount silicon and run a motion-cycle stability check:

```
py tools\cycle_test.py --cycles 3 --frames 20 --moves both --tag si
```

The log reports each band's change relative to the baseline; if one motion drops the signal by $> 2\%$, isolate it (`--moves arm`, `--moves sample`, ...). Repeat with the white board (`--tag white`) to separate a sample-mounting problem from a detector-arm repeatability problem.

4.4 D. Shutdown

Use the GUI's **Disconnect**/exit or let a script finish normally — both record the motor state and close the shutter. Afterwards do not plug or unplug anything on that USB hub.

5 The graphical interface

5.1 Pages

A sidebar switches between five pages:

1. **Instrument** — connect/disconnect, a quick-start checklist, live motor state and notes.
2. **Motors & shutter** — manual moves and the shutter.
3. **Spectrometer** — open the device, set or auto-calibrate the integration time, read a frame, live view, and the **Recover** button.
4. **Measurement** — build and run a sequence; live spectrum preview and a completed-acquisition table.
5. **Analysis** — compute and export reflectance from stored sessions.

5.2 Status bar and the emergency shutter

A persistent status bar shows the serial port, spectrometer, integration time, shutter, detector-arm angle and sequence state. A red **Close shutter** button is available from every page.

5.3 Keyboard shortcuts

[Ctrl/Cmd + 1..5]	switch to page 1–5
[Ctrl/Cmd + R]	run the current sequence
[Esc]	abort the running sequence
[F5]	query all instrument info
[Ctrl/Cmd + L]	toggle the log drawer
[Ctrl/Cmd + Shift + S]	save the log to a file

The emergency **Close shutter** button has deliberately no shortcut (so it cannot be triggered by accident). Manual controls are locked while a sequence runs.

6 Running a measurement

6.1 Connect

On the **Instrument** page, connect the bus and open the spectrometer. On connect, OptiComp2 sets the detector-arm and sample-stage speeds to 50 % and compares the stage state against the baseline; resolve any anomaly (Section 4) before measuring.

6.2 Reference calibration (with auto integration time)

Build a **reference calibration** step on the Measurement page. It moves the detector arm to its zero and the sample to the calibration angle (80°), opens the shutter, and calibrates the integration time *automatically* in both S and P, keeping the smaller of the two:

- The peak is driven toward 85 % of full scale (accept band 78–92 %) by linear extrapolation, halving on saturation, in at most eight steps.
- A dim target legitimately needs a long exposure (silicon at 80° in P calibrates at tens of seconds); each long read is announced so it is not mistaken for a hang.
- If the band cannot be reached even at the 60s hardware ceiling, the calibration **aborts with a diagnostic** (reference in place? lamp warmed up? fibre? shutter?) rather than using a bad value.

Lamp-stability gate. By default, the reference calibration waits for the lamp to settle before calibrating: the active-band mean must hold within 0.5 % for five consecutive reads (it warns and proceeds after a timeout, never blocking). This prevents lamp drift between the reference and sample sessions from biasing the reflectance. Three toggles on the Measurement page’s progress row control the pre-run behaviour: **Reconnect** (reopen the spectrometer), **Zero** (return all stages to zero), and **Stabilise** (the lamp-stability gate).

6.3 Dark, single angle, scan and double beam

- **Dark** — a shutter-closed frame stored per integration time (`dark_997ms.csv`), matched to the spectrum during analysis.
- **Single angle** — one polarisation and angle.
- **Scan** — an angle range in one or both polarisations; the gatekeeper enforces $0 \leq \text{start} < \text{stop} \leq 80$ and $\text{step} \geq 1$.
- **Double beam (DB)** — the port-cap exchange step at a fixed 1000 ms integration time, with its own dark; the sequence pauses for the physical exchange.

Run with [Ctrl+R]. Each acquisition is written to a session directory and appended to its `manifest.json`; the completed-acquisition table updates live. If a run fails or is aborted, the shutter is closed automatically.

7 Analysis

On the **Analysis** page, select a sample session, a reference session and a standard, then compute. Reflectance uses the double-beam substitution relation

$$R_x = \frac{S_x - S_d}{S_y - S_d} \cdot \frac{S_{cy} - S_d}{S_{cx} - S_d} \cdot R_y,$$

where x is the sample, y the reference, S_d the matching dark, and the S_c terms the double-beam channel. Key points:

- Darks are matched by integration time; if they differ, counts are normalised per millisecond and a note is recorded.

- Active pixels are 254–2030 inclusive (1777 pixels).
- Standards: a constant (white reference), a silicon Fresnel table (real n ; invalid below 380 nm and exported as NaN), a BK7 Sellmeier model and a slab back-surface inversion.
- Saturated pixels and invalid wavelengths are exported as NaN, never as a spurious reflectance.

Results can be exported to CSV and the figure saved.

8 Command-line tools

All tools live in `tools\` and are run with `py tools\<name>.py`.

Tool	Purpose
<code>restore_stages.py</code>	Read-only bus report; <code>--safe --arm</code> restores the reference under supervision (needs an interactive terminal to type YES).
<code>usb_reset.py</code>	Probe, or PnP-restart the spectrometer USB device (administrator).
<code>shutter_close.py</code>	Close and verify the shutter (position 0) after a hard-killed script.
<code>monitor.py</code>	Unattended stability monitor: continuous reads at a fixed geometry logged to CSV with each band's change relative to the first frame.
<code>cycle_test.py</code>	Motion-cycle test (<code>sample/arm/scan/exchange/ both</code>), a dark per cycle, automatic recovery.
<code>ell_probe.py</code> , <code>spec_probe.py</code>	Read-only probes of the bus and the spectrometer.

Stopping an unattended script

`monitor.py` and `cycle_test.py` check for `logs\STOP` every frame. To finish early, create the file:

```
New-Item C:\OptiComp2\logs\STOP
```

The script then closes the shutter, saves its data, records the state and exits. Do *not* use `taskkill` or close the window — that skips the cleanup and can leave the shutter open.

9 Recovery and troubleshooting

Symptom	Action
Spectrometer read returns <code>-99</code> / device count 0 (DLL hung).	Press Recover on the Spectrometer page (or let monitor/cycle recover): it reopens the DLL, then PnP-restarts the spectrometer USB device. Needs the elevated launcher.
Detector arm reports <code>GS02</code> (home failed, zero lost).	Do <i>not</i> re-home casually. Follow Section 4A with the fibre slack and someone watching.
A move seems to do nothing.	OptiComp2 verifies position and resends up to three times; if it still cannot confirm, it raises. Check the stage mechanically.
Auto integration time aborts “no light” / “too dim”.	Check the reference is in place, the lamp is warm, the fibre is connected and the shutter opened.
Stage-state anomaly banner on connect.	A module moved between sessions (often an auto-home after a power event). Verify positions and restore the reference (Section 4A) before measuring.
Shutter left open after a crash.	<code>py tools\shutter.close.py</code> (does exactly one thing: close and verify position 0).

10 Data and file conventions

- One `manifest.json` per session directory, written atomically; an unreadable manifest is moved aside as `manifest.json.corrupt_<time>` rather than overwritten.
- Darks are stored per integration time (`dark_997ms.csv`, `dark_db_1000ms.csv`); the analysis picks the dark whose integration time matches the spectrum.
- Spectra are stored as raw counts; smoothing is display-only.
- The stage-state record `data\stage_state.json` holds each module’s last position, status and speed, and is the baseline for the anomaly check.

A Geometry and configuration reference

Constant	Value
Bus addresses (shutter / polariser / arm / sample)	0 / 1 / 2 / 3
Polariser angles (P, S)	146°, 236°
Detector-arm zero (<code>SYSTEM.ZERO</code>)	44°
Detector-arm double-beam / exchange	124° / 150°
Sample zero / VAR offset	103° / 105° (stage = $\theta + 105$)
Angle range / minimum step	0–80° / 1°
Soft limits (arm, sample)	0–200°
Stage speeds applied on connect	50 %
Protected home	detector arm (address 2)
State tolerance	0.5°
Double-beam integration time	1000 ms
Lamp-stability gate	0.5 % for 5 reads, ≤ 60 -read timeout
Spectrometer USB VID (for PnP restart)	VID_16A3

Golden rules. (1) The original OptiComp is closed. (2) The fibre arm is never homed casually. (3) No USB replug on the instrument hub while the arm is away from zero — use `usb_reset.py`. (4) Never hard-kill a script — use the `STOP` file, and `shutter_close.py` if you already did. (5) Resolve any stage-state anomaly before measuring.